



Minijuego

GDD · Arcade Space Shooter 2D · Unity / C# · Versión 1.0

1. Resumen

Atributo	Especificación
Género	Arcade Space Shooter 2D / Retro Physics Action
Plataforma	macOS — Apple Silicon e Intel — y WebGL
Modo y cámara	Un jugador · Top-Down 2D ortográfico
Audiencia	Jugadores casuales y aficionados al arcade retro

Premisa: el jugador pilota una nave dentro de un cinturón de asteroides, usando propulsión inercial y disparos frontales para esquivar amenazas, sobrevivir y lograr la máxima puntuación.

Pilares: control físico satisfactorio, disparos precisos y dificultad progresiva.

2. Iteraciones y arte

Iteración 1 — Base del profesor

Movimiento básico de la nave, disparos, configuración del `Player`, creación de meteoritos y detección de colisiones.

Iteración 2 — Mejoras propias

- Menú de pausa con `PauseMenu.cs` y tecla `Esc`.
- Spawner calculado según la visión real de la cámara mediante `Camera.main.ViewportToWorldPoint()`.
- Mejora de aceleración, rotación e inercia de la nave.
- Límites envolventes: la nave reaparece por el extremo contrario al salir de la pantalla.

Dirección artística

Se utilizan únicamente los sprites 2D proporcionados por el profesor: **bala**, **fondo espacial**, **meteorito** y **nave**. No se han añadido recursos externos. Su estilo parece basado en **pixel art** y mantiene una estética retro coherente con el género arcade.

3. Mecánicas principales

- **Player** — **Player.cs** : propulsión con `Rigidbody.AddForce(thrust * transform.right)`, rotación sobre Z, disparo desde **BulletSpawner** usando el prefab **Bullet**, amortiguación mediante **Linear Damping** y derrota al colisionar con **Enemy**. La nave reaparece por el lado contrario al superar un límite.
- **Meteoritos** — **EnemySpawner.cs** : aparecen por encima de la zona visible, dentro de los límites X de la cámara; descienden con velocidad y dirección variables, giran mediante torque y se destruyen al abandonar la pantalla o superar su tiempo de vida.
- **Proyectiles** — **Bullet.cs** : avanzan en línea recta, destruyen meteoritos al impactar, incrementan la puntuación y se autodestruyen tras su tiempo máximo.
- **Puntuación y HUD**: **Player.SCORE** aumenta al destruir asteroides, sin una cantidad fija por impacto. El **Canvas** usa **Screen Space - Overlay** y muestra **PUNTOS: [SCORE]**; el menú utiliza el panel **Panel**.

4. Controles

Acción	Teclado	Alternativa
Acelerar	W	Flecha arriba
Rotar izquierda / derecha	A / D	Flechas izquierda / derecha
Disparar	Barra espaciadora	Click izquierdo
Abrir pausa	Esc	—

5. Parámetros del Inspector

Valores ajustados según la configuración observada en Unity.

Componente	Parámetro	Valor
Player	Thrust Force / Rotation Speed	7.5f / 100.0f

Componente	Parámetro	Valor
Rigidbody (Ship)	Mass / Linear Damping / Angular Damping	1 / 1.5f / 0.05f
Rigidbody (Ship)	Gravity / Kinematic	Desactivados
Rigidbody (Ship)	Interpolate / Collision Detection	None / Discrete
Transform (Ship)	Scale	X 0.2 / Y 0.2 / Z 1
Capsule Collider	Radio / altura / dirección	0.3 / 6 / Y-Axis
Player	Gun / Bullet Prefab	BulletSpawner / Bullet
Bullet	Speed / Max Life Time	12.0f / 2.5f
Enemy Spawner	Spawn / incremento	30 por minuto / +1.5 por ciclo
Asteroid	Velocidad min. / máx. / Torque Max	2.5f / 5.5f / 40.0f
Canvas	Render Mode	Screen Space - Overlay
Canvas Scaler	Modo / resolución / Match / PPU	Scale With Screen Size / 1920 x 1080 / 1 - Height / 100
PauseMenu	Pause Menu UI	Panel

6. Arquitectura de scripts

```
Scripts/
├─ Player.cs      # Movimiento, físicas, disparo y colisión fatal
├─ EnemySpawner.cs # Límites de cámara y gestión de meteoritos
├─ PauseMenu.cs   # Menú de pausa con Esc
└─ Bullet.cs      # Movimiento, vida e impacto de proyectiles
```

7. Roadmap

Completado: movimiento físico, aceleración y rotación; disparos y colisiones; spawner calculado según cámara; derrota y reinicio; puntuación básica; límites envolventes y menú de pausa.

Mejoras futuras no implementadas:

- Object Pooling para reutilizar proyectiles y mejorar el rendimiento.

- División de asteroides al recibir el impacto de una bala.
- Sistema de vidas del jugador.
- Partículas de propulsión y explosión.
- Efectos de sonido de láser, explosión y propulsor.
- Guardado de la puntuación máxima mediante `PlayerPrefs`.

8. Validación y autoría

La nave debe responder de forma predecible; los meteoritos deben aparecer dentro del ancho visible; las colisiones, puntuación, derrota y reinicio deben funcionar sin errores; y la dificultad debe aumentar progresivamente sin perder legibilidad ni rendimiento.

Documento realizado por: Juan Outerelo Sánchez · **Matrícula:** 240283 · **DNI:** 06014164D

Elaborado en Notion con ayuda de su inteligencia artificial a partir del proyecto de Unity, el material inicial del profesor y las mejoras del alumno.